

DEPARTMENT OF COMPUTER SCIENCE



University of
Reading

CS1IP: Imperative Programming

Week 8: Top-down Design

Learning Outcomes

- What is to Top-Down Design?
- How to implement a text-based game?

What is to Top-Down Design?

What is a program?

A sequence of **instructions** that is **executed** by a **computer**.

When number of instructions grown with the problem complexity.

One way to handle the complexity is break the problem in sub-problems.

What is to Top-Down Design?

It is a problem-solving approach

- Starts with the highest level of abstraction
- Breaks down complex problems into smaller, manageable components
- Implement the smaller tasks and combine them.



Why is it important?

Simplifies complex problems and promotes:

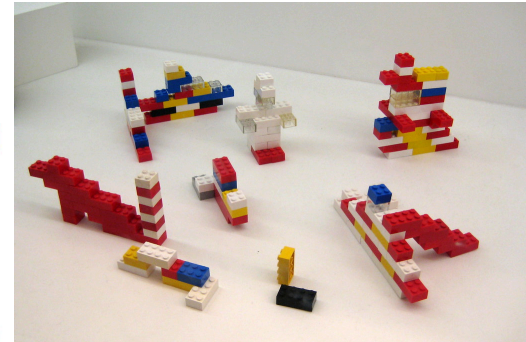
- **Modularity**
 - Facilitates code reuse and maintainability
- **Structured Approach:**
 - Ensures clarity and organisation
 - Provides a clear roadmap for implementation
- **Enhanced Debugging:**
 - Easier to identify and fix issues in isolated components
- **Scalability:**
 - Adapts well to large, complex projects



How do a Top-Down Design?

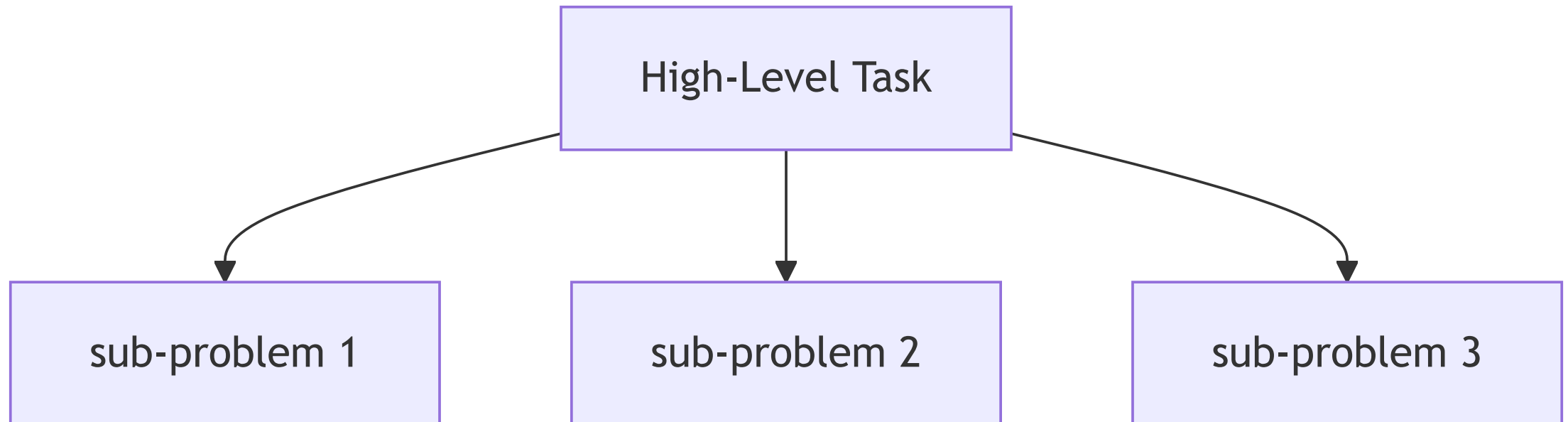
Use Decomposition

- Step 1 - Define the High-Level Task
- Step 2 - Divide the Problem into Sub-problems
- Step 3 - Refine Each Subproblem



How do a Top-Down Design?

You can Diagram



How do a Top-Down Design?

You can do when programming

- Use **TODO** comments
- Use **Java functions**:
 - Primary unit of functionality
 - Reuse functions
 - Make the main part your program be simple and with few function calls

```
void highLevelProgram(){  
    // TODO sub-problem 1  
    // TODO sub-problem 2  
    // TODO sub-problem 3  
}  
  
highLevelProgram()
```

Keep mind

Diagrams help to describe sub-problems visually

TODO Comments help to describe sub-problems inside your program

Example Rectangular Box

Create a Rectangular Box using for give hight

```
*****  
*      *  
*      *  
*****
```

Which steps can we do ? 🤔

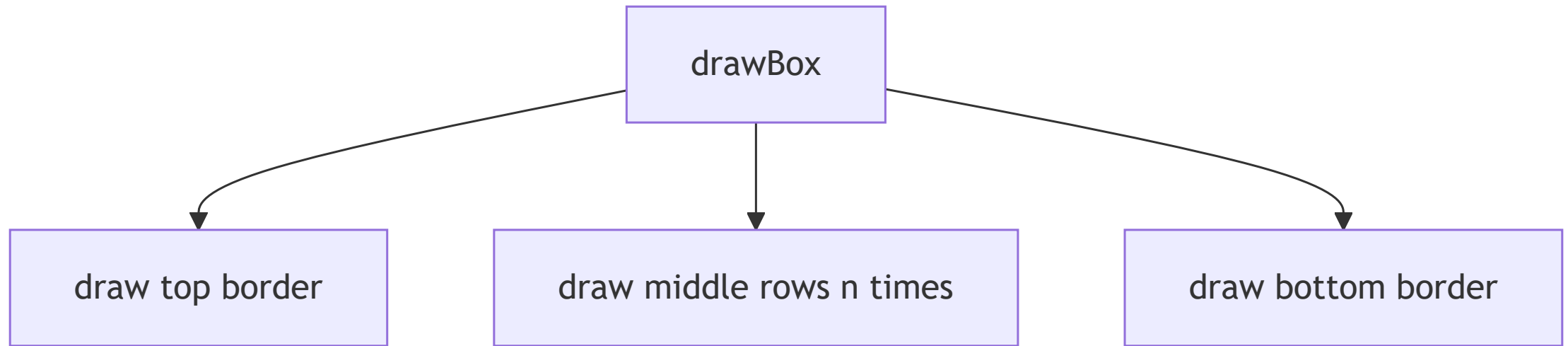
Step 1 - Define the High-Level Task (Diagram)

```
*****  
*      *  
*      *  
*****
```

drawBox

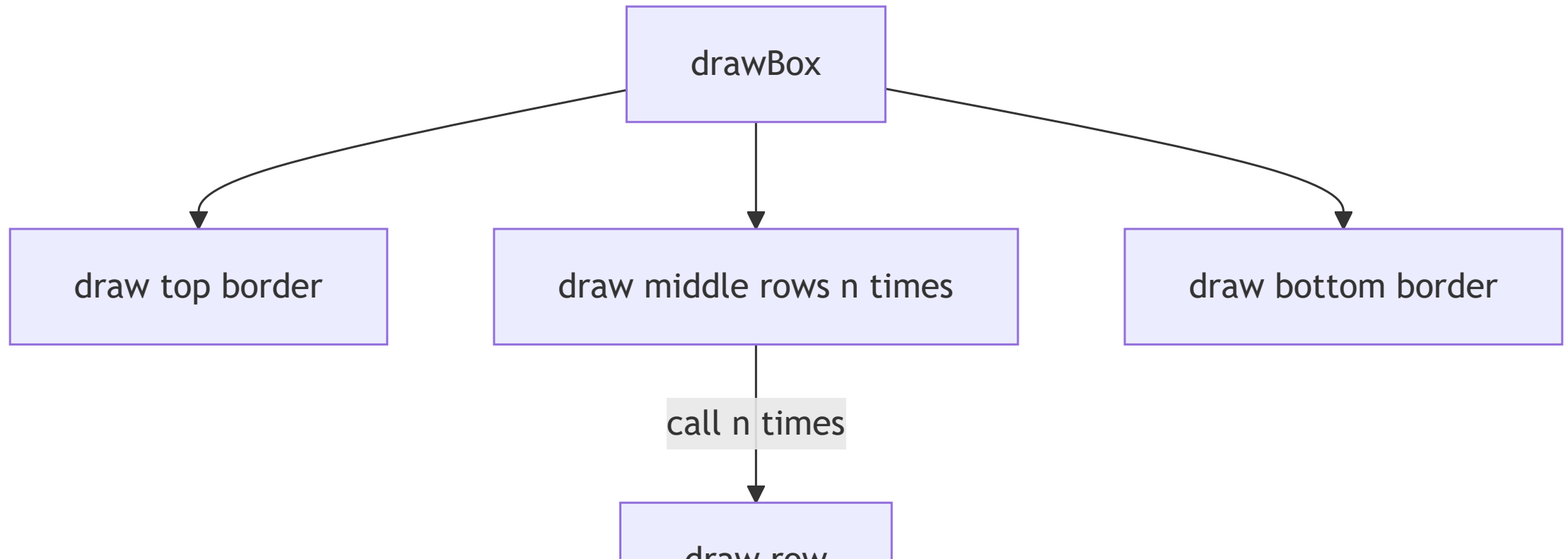
Step 2 - Divide the Problem into Sub-problems (Diagram)

```
*****  
*      *  
*      *  
*****
```



Step 3 - Refine Each Subproblem (Diagram)

```
*****  
*      *  
*      *  
*****
```



How can we do with TODO comments?

Top-down Design Rectangular Box

Step 1 - Define the High-Level Task (TODO comments)

```
void drawBox(int n) {  
    // TODO  
}  
drawBox(10);
```

Top-down Design Rectangular Box

Step 2 - Divide the Problem into Sub-problems (TODO comments)

```
void drawBox(int n) {  
    // Draw the top border.  
    // Draw the middle rows n times.  
    // Draw the bottom border.  
}  
drawBox(10);
```

Top-down Design Rectangular Box

Step 2 - Divide the Problem into Sub-problems (TODO comments)

```
void drawBox(int n) {  
    drawTopBorder();  
    drawMiddleRows(n);  
    drawBottomBorder();  
}  
drawBox(10);
```

Top-down Design Rectangular Box

Step 3 - Refine Each Subproblem (TODO comments)

```
void drawTopBorder() {  
    // TODO  
}  
  
void drawBottomBorder() {  
    // TODO  
}  
  
void drawRow(){  
    // TODO  
}  
  
void drawMiddleRows(int n) {  
    // TODO call drawRow n times  
}  
  
void drawBox(int n) {  
    drawTopBorder();  
    drawMiddleRows(n);  
}
```

Top-down Design Rectangular Box (implementation)

`drawBorder()`

- Print a line of asterisks (`*****`).

`drawBottomBorder()`

- Print a line (`*   *`) `n` times

`drawBottomBorder()`

- Print a line of asterisks (`*****`).

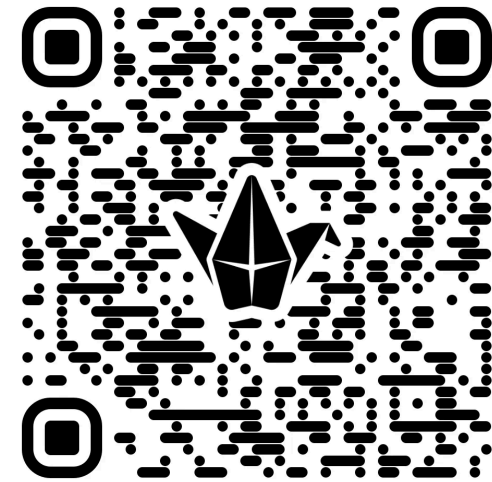
Top-down Design Rectangular Box (implementation)

```
void drawTopBorder() {  
    System.out.println("*****");  
}  
  
void drawBottomBorder() {  
    System.out.println("*****");  
}  
  
void drawRow(){  
    System.out.println("*      *");  
}  
  
void drawMiddleRows(int n) {  
    for (int i = 0; i < n; i++) {  
        drawRow();  
    }  
}  
  
void drawBox(int n) {  
    drawTopBorder();  
    drawMiddleRows(n);  
    drawBomBorder();  
}  
drawBox(10);
```

Quiz 1 ([click here](#) or scan QRCode)

Which of the following is the primary benefit of Top-Down Design?

- a. It allows programmers to jump directly in programming without planning.
- b. It ensures the code is written only in Java.
- c. It breaks complex problems into manageable sub-problems.
- d. It is only suitable for simple programs.



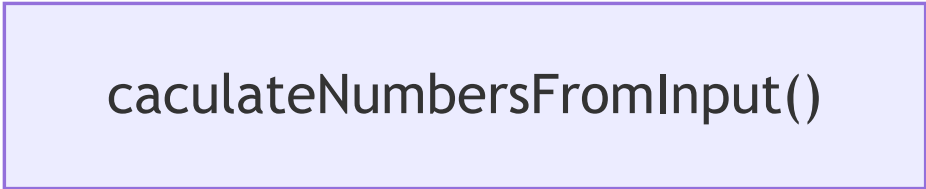
Example: Top-down Design Calculator

Top-down Design Calculator

Problem Statement:

Calculate operation of two numbers from user input. Example : `1 + 1` and `2 - 1`

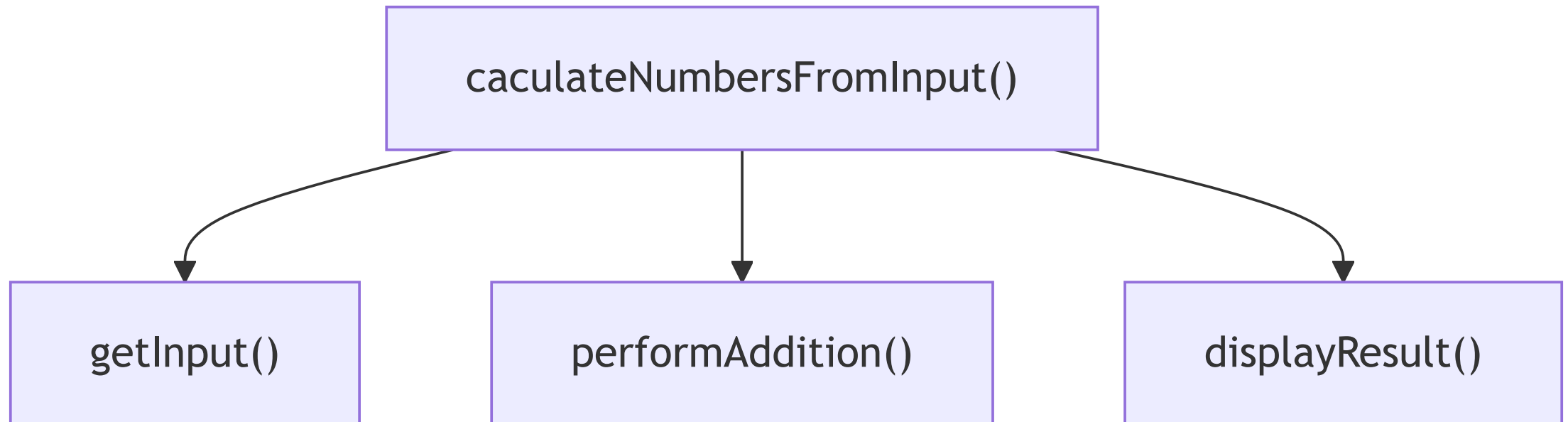
Step 1 - Define the High-Level Task (Diagram)



```
caculateNumbersFromInput()
```

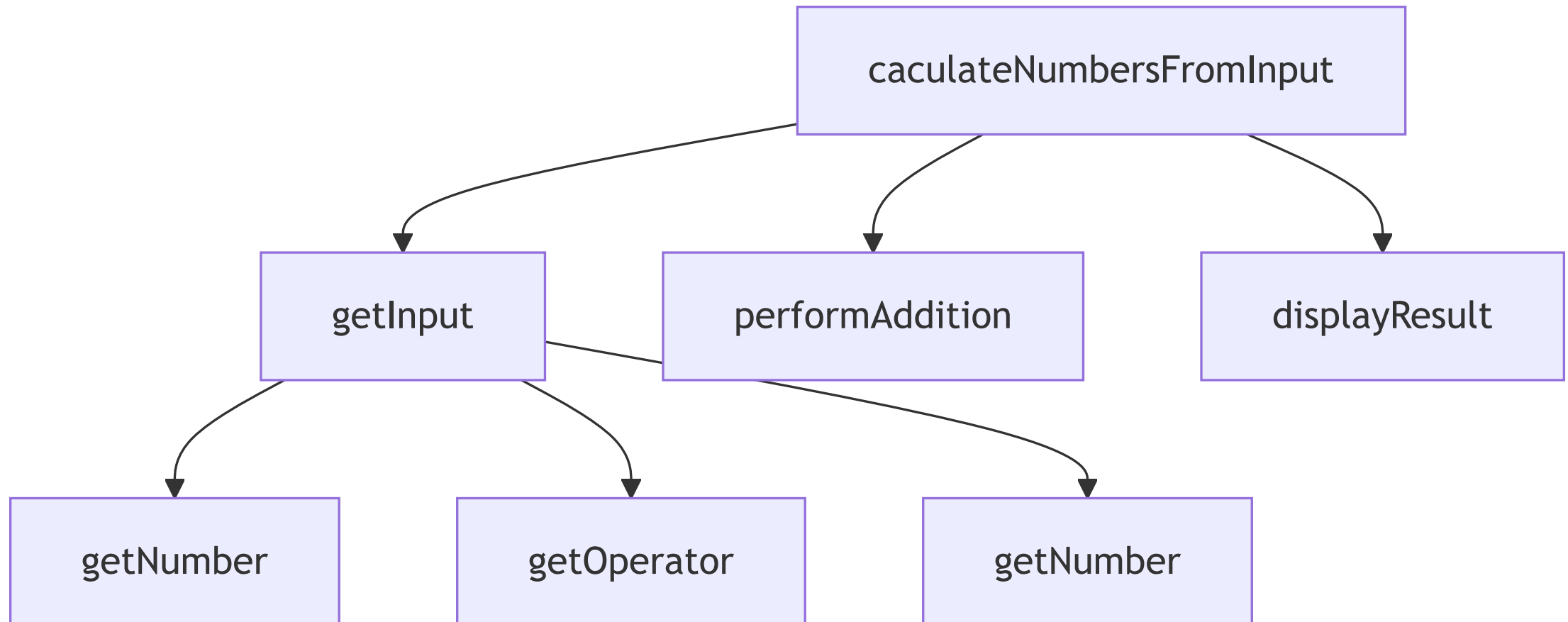
Top-down Design Calculator

Step 2 - Divide the Problem into Sub-problems (Diagram)



Top-down Design Calculator

Step 3 - Refine Each Subproblem (Diagram)



Top-down Design Calculator

Step 1 - Define the High-Level Task (TODO comments)

```
void calculateNumbersFromInput(){  
    //calculation for operations of two numbers  
}
```

Top-down Design Calculator

Step 2 - Divide the Problem into Sub-problems (TODO comments)

```
void calculateNumbersFromInput(){  
    // TODO getInput  
  
    // TODO performAddition  
  
    // TODO displayResult  
}
```

Top-down Design Calculator

Step 3 - Refine Each Subproblem (Diagram)

```
void calculateNumbersFromInput(){  
    // TODO get first int  
    // TODO get get operand  
    // TODO get second int  
  
    // TODO performAddition  
  
    // TODO displayResult  
}
```

Top-down Design Calculator (implementation)

```
void calculateNumbersFromInput(){  
  
    // TODO getInput  
    Scanner scan = new Scanner(System.in);  
    int num1 = scan.nextInt(); // get first int  
    char operand = scan.next().charAt(0); // get operand  
    int num2 = scan.nextInt(); // get second int  
    int result;  
  
    // TODO performAddition  
  
    // TODO displayResult  
}
```

Top-down Design Calculator (implementation)

```
void calculateNumbersFromInput(){
    // getInput
    Scanner scan = new Scanner(System.in);
    int num1 = scan.nextInt(); // get first int
    char operand = scan.next().charAt(0); // get operand
    int num2 = scan.nextInt(); // get second int

    // performAddition
    int result;
    if (operand == '+'){
        result = num1-num2;
    } else if(operand == '-'){
        result = num1+num2;
    }

    // TODO displayResult
}
```

Top-down Design Calculator (implementation)

```
void calculateNumbersFromInput(){
    // getInput
    Scanner scan = new Scanner(System.in);
    int num1 = scan.nextInt();
    char operand = scan.next().charAt(0);
    int num2 = scan.nextInt();
    int result;

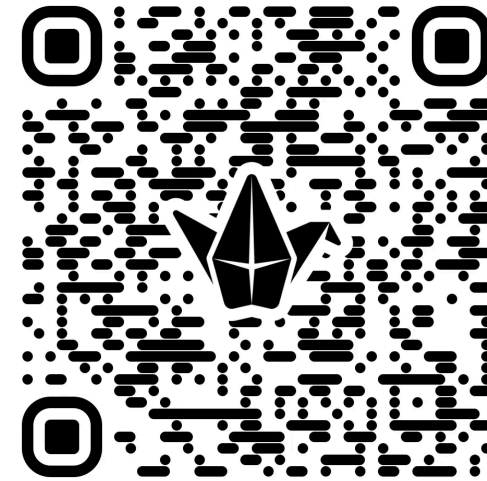
    // performAddition
    if (operand == '+'){
        result = num1-num2;
    } else if(operand == '-'){
        result = num1+num2;
    }

    // displayResult
    System.out.println("The result is "+ result);
}
```

Quiz 2 ([click here](#) or scan QRCode)

In Top-Down Design, what is the primary purpose of using comments and functions in code?

- a. To replace the need for writing a complete program.
- b. To document only the sections of code that are hard to understand.
- c. To divide a program into smaller, understandable parts and make the code modular.
- d. To ensure that the program compiles without errors.



Take home message from Calculator

You not always will need use functions to divide sub-problems

But you always need to write sub-problems and **TODO comments**

Dungeon Game

Dungeon Game

- The player can open doors.
- Behind each door can be either:
 - A **Monster**: Must be defeated randomly.
 - A **Treasure**: Increases the player's score.
- The goal is to accumulate as much score as possible.



Dungeon Game

Step 1 - Define the High-Level Task (Diagram)

Present the player with a choice to open the door or quit and update the game state based on the outcome (monster or treasure)

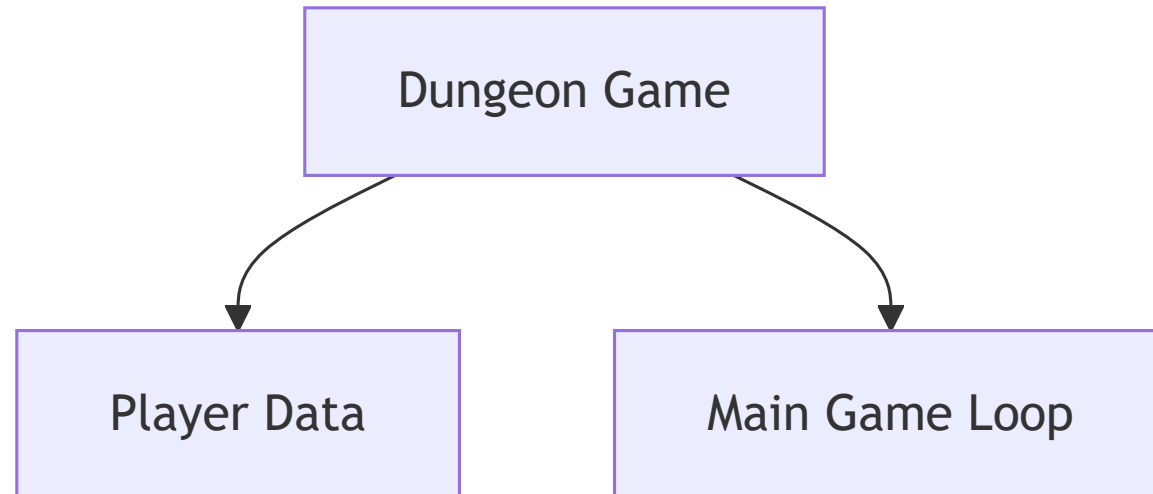


Dungeon Game

Dungeon Game

Step 2 - Divide the Problem into Sub-problems (Diagram)

- **Player Data:** Tracks the player's score.
- **Main Game Loop:** Controls the game's flow and ask for player action.



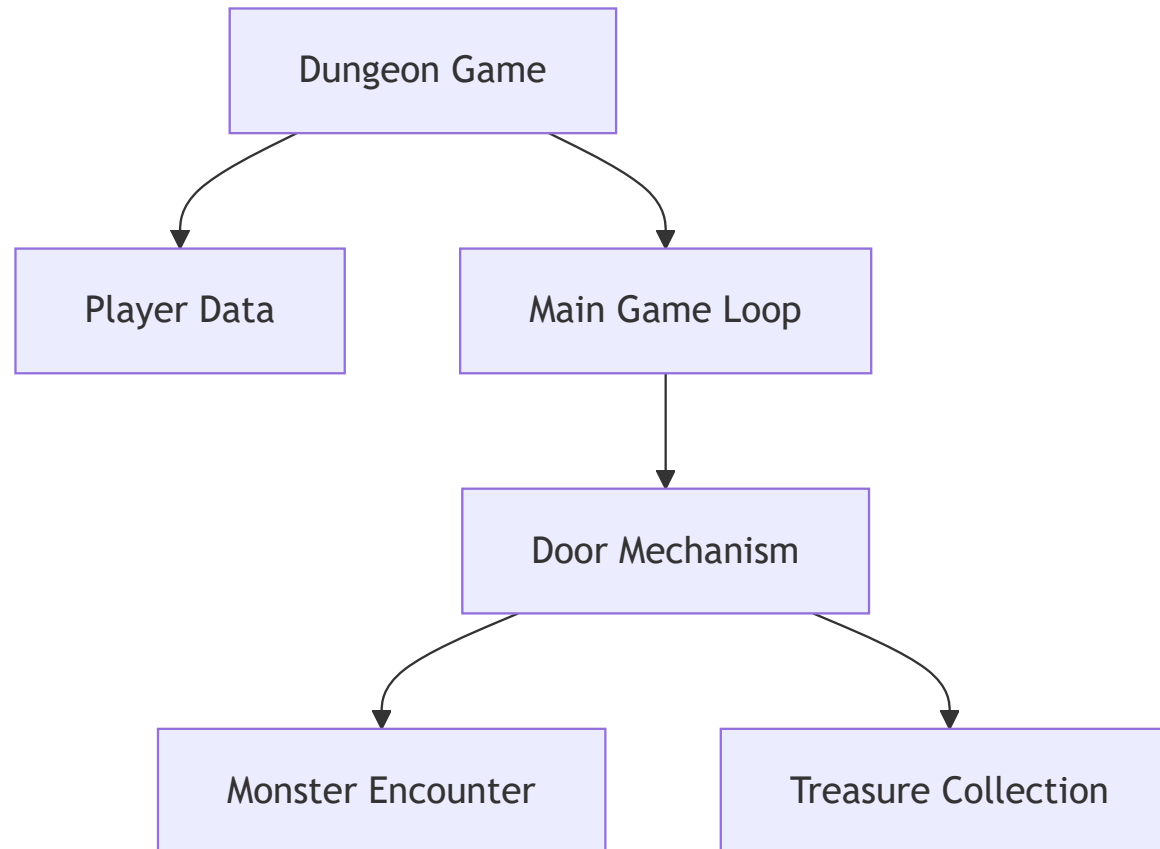
Dungeon Game

Step 3 - Refine Each Subproblem (Diagram)

- **Door Mechanism**
 - **Monster Encounter:** Randomly decides if the player defeats the monster.
 - **Treasure Collection:** Increases the player's score.

Dungeon Game

Step 3 - Refine Each Subproblem (Diagram)



Dungeon Game: TODO comments and implementation

Dungeon Game

```
// TODO Player Data  
// TODO Main Game Loop
```

Dungeon Game

```
// TODO Player Data  
int score = 0;  
boolean isPlaying = true;  
  
// TODO Main Game Loop
```

Dungeon Game

```
// Player Data
int score = 0;
boolean isPlaying = true;

boolean function doorMechanism(){
    // TODO return true if survive the door
}

// TODO Main Game Loop
while (isPlaying) {
    // TODO Controls the game's flow and ask for player action.
    // TODO if choice open door call doorMechanism
}
```

Dungeon Game

```
...
boolean function doorMechanism(){
    // TODO return true if survive the door
}

while (isPlaying) {
    // Controls the game's flow and ask for player action.
    System.out.println("You see a door. Do you want to open it? (yes/no)");
    Scanner scan = new Scanner(System.in);
    String choice = scan.nextLine();

    // if choice open door call doorMechanism
    if (choice.equals("yes")) {
        isPlaying = doorMechanism(score);
    } else {
        isPlaying = false;
    }
    System.out.println("Current Score: " + score);
}

System.out.println("Game Over! Final Score: " + score);
```

Dungeon Game: doorMechanism

- Door Mechanism can be broken down into:
 - Randomly determine if a monster or treasure behind the door
 - If it's a monster, trigger the monster encounter.
 - If it's a treasure, increase the player's score.

Dungeon Game: doorMechanism

```
boolean function doorMechanism(){  
    // TODO Randomly determine if a monster or treasure behind the door  
    // TODO If it's a treasure, increase the player's score.  
    // TODO If it's a monster, trigger the monster encounter.  
    // TODO return true if survived  
}
```

Dungeon Game: doorMechanism

```
int function treasureCollection(){
    // TODO
}
boolean function monsterEncounter(){
    // TODO
}
boolean doorMechanism() {
    // TODO
}
boolean function doorMechanism(){
    // TODO Randomly determine if a monster or treasure behind the door
    // TODO If it's a treasure, call treasureCollection
    // TODO If it's a monster, call monsterEncounter
    // TODO return true if survived
}
```

Dungeon Game: doorMechanism

```
boolean function monsterEncounter(){
    // TODO
}
int function treasureCollection(){
    // TODO
}
boolean doorMechanism() {
    // Randomly assigns a monster or treasure behind each door.
    Random random = new Random();
    int outcome = random.nextInt(2); // 0 or 1
    if (outcome == 0) {
        // call treasureCollection()
        System.out.println("You found a treasure!");
        score += treasureCollection(); // global variable score
    } else {
        // call monsterEncounter()
        System.out.println("A monster appears!");
        if (monsterEncounter()) {
            System.out.println("You defeated the monster!");
        } else {
            System.out.println("The monster defeated you!");
            return false;
        }
    }
}
return true;
}
```

Dungeon Game: treasureCollection()

```
int treasureCollection() {  
    Random random = new Random();  
    int treasureValue = random.nextInt(100)+1;  
    System.out.println("You collected a treasure worth " + treasureValue + " points!");  
    return treasureValue;  
}
```

- **Random Treasure Value:** Each treasure has a random value between 1 and 100.
- **Score Update:** The player's score is increased by the treasure's value.

Dungeon Game: `monsterEncounter()`

```
boolean monsterEncounter() {  
    Random random = new Random();  
    int fightOutcome = random.nextInt(2); // 0 or 1  
    return fightOutcome == 1; // 50% chance to win  
}
```

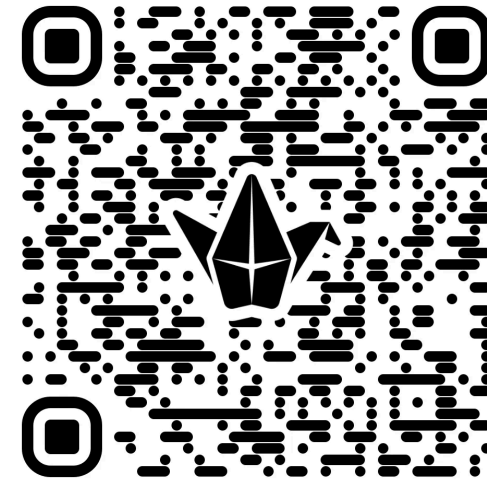
- **Random Fight Outcome:** The player has a 50% chance of defeating the monster.
- **Victory:** The game continues.
- **Defeat:** The game ends.

Quiz 2 ([click here](#) or scan QRCode)

Which line best represents the nature of Top-Down Design?

```
while (isPlaying) {  
    System.out.println("Choose an action: (open/quit)");  
    String action = scanner.nextLine();  
    if (action.equals("open")) {  
        boolean survived = doorMechanism();  
        if (!survived) {  
            isPlaying = false;  
        }  
    } else if (action.equals("quit")) {  
        isPlaying = false;  
    }  
    System.out.println("Current Score: " + score);  
}  
System.out.println("Game Over! Final Score: " + score);
```

- a. boolean survived = doorMechanism();
- b. System.out.println("Game Over! Final Score: " + score);
- c. while (isPlaying) { ... }
- d. String action = scanner.nextLine();



Review Learning Outcomes

Common Pitfalls and How to Avoid

- **Over-Decomposition:**
 - Avoid making methods too granular
- **Poor Abstraction:**
 - Keep the high-level design clear and logical
- **Skipping Design:**
 - Avoid jumping directly in programming. Plan first!
- **Not Refactoring:**
 - Continually refine and improve the code structure

Review Learning Outcomes

What is to Top-Down Design?

- Review key points on Top-Down Design
- Breaking down tasks leads to modular and maintainable code.
- Top-down design helps manage complexity.

How to implement a text-based game?

- functions to every action and sub actions
- main loop to capture user actions

DEPARTMENT OF COMPUTER SCIENCE



Thank you! 😊

Do you have any questions 🙋 ?

The next lecture will cover: Big-O and search